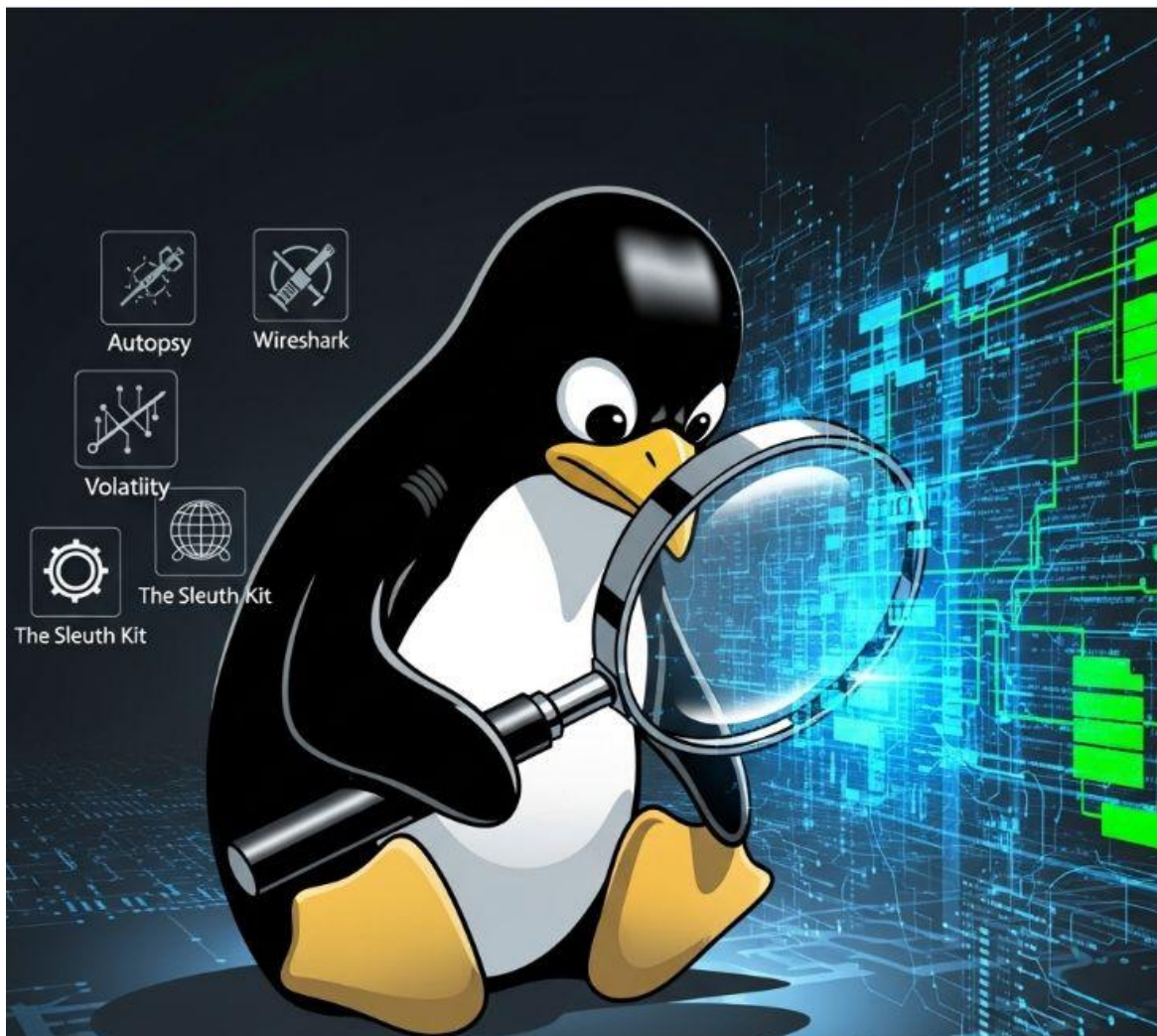


Análisis Forense Informático

Unidad 8. Forense en Linux



Índice

1	Introducción.....	2
2	El sistema de ficheros de Linux EXT2, 3, 4	2
2.1	El Superblock	3
2.2	Los inodos	4
2.3	¿Qué ocurre cuando se crea un fichero?	6
2.4	¿Cómo se elimina un fichero?	6
2.5	Recuperación de archivos	6
3	Artefactos Linux.....	7
3.1	La jerarquía de directorios Linux	8
3.2	Timestamps en Linux	10
3.3	Información básica de un sistema Linux.....	11
3.4	Análisis de logs	14
3.5	Servicio SSH	17
3.6	Historial de comandos y acceso a ficheros	17
3.7	Entorno gráfico.....	18
3.8	Backdoors	18
3.9	Persistencia	18
4	Triaje en sistemas Linux	19
5	Ejemplo práctico	23
5.1	Obtención de información de un sistema vivo	23
5.2	Análisis de imagen de un sistema Linux	25

1 Introducción.

La informática forense aplicada a sistemas Linux se centra en el análisis técnico y metodológico de estos sistemas cuando han sido objeto de un incidente de seguridad, un uso indebido o una investigación interna o judicial. El objetivo principal es reconstruir los hechos ocurridos en el sistema, identificar acciones realizadas por usuarios o procesos y obtener evidencias digitales válidas, manteniendo en todo momento su integridad y trazabilidad.

Los sistemas Linux presentan características propias que influyen directamente en el análisis forense, como su estructura de archivos, los mecanismos de permisos y usuarios, la gestión de procesos, los registros del sistema (logs) y los distintos sistemas de archivos utilizados. El conocimiento detallado de estos elementos resulta esencial para interpretar correctamente la información obtenida durante una investigación.



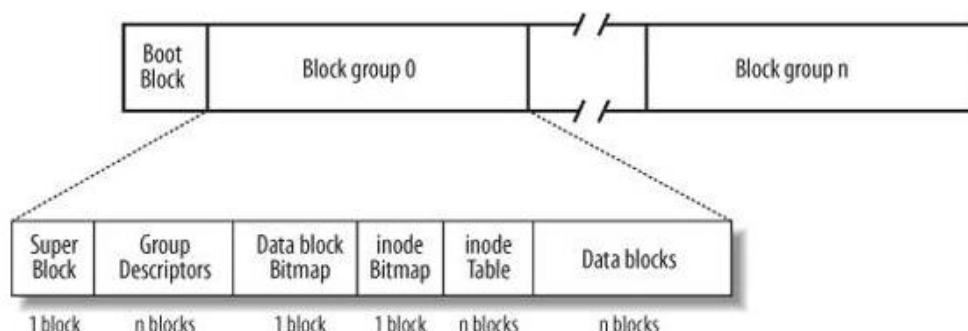
En el ámbito forense, el análisis de un sistema Linux puede incluir el estudio de cuentas de usuario, tareas programadas, servicios en ejecución, archivos de configuración, historial de comandos, registros de autenticación y eventos del sistema. Estas evidencias permiten detectar accesos no autorizados, escaladas de privilegios, persistencia de malware o manipulaciones del sistema.

Primero empezaremos por recordad el funcionamiento del sistema de ficheros de Linux

2 El sistema de ficheros de Linux EXT2, 3, 4

El sistema de archivos (File System o FS) más utilizado en Linux es EXT (el número es la versión). Cada partición en EXT contiene:

- Un bloque de arranque opcional, ya que puede que la partición no sea de arranque.
- M bloques para la tabla de descriptores de grupos (si la partición es muy grande, se hacen grupos de bloques).
- Un bloque para el Superbloque que define qué tamaño tienen el resto de zonas.
- Un bloque para el bitmap de bloque (un bit indica para cada bloque si está libre u ocupado).
- Un bloque para el bitmap de Inodos (un bit indica para cada Inodo si está libre u ocupado).
- N bloques por la tabla de Inodos (bloques que contienen información de los archivos).
- N bloques de datos (donde cada uno puede estar libre u ocupado).



2.1 El Superblock

El **Superblock** contiene una descripción del tamaño y organización del sistema de archivos y permite gestionarlo.

Cuando se monta el sistema de ficheros, normalmente sólo se lee el Superblock del grupo de bloques 0, pero cada grupo de bloques contiene un duplicado del superblock por si ocurre el caso de tener errores en el sistema de ficheros. El Superblock contiene la siguiente información:

- **Número mágico**, para indicar la versión. Por ejemplo: Ext2 es 0xEF53.
- **Contador de montajes que se han realizado y máximo valor de montajes**: cuando se alcanza el máximo de este contador, se realiza un check automático del sistema de ficheros.
- **Número de grupo de bloques** que contiene esta copia de Superblock.
- **Tamaño de bloque**, por ejemplo 4096 Bytes
- **Número de bloques que tiene cada grupo**: este valor se fija cuando se crea el sistema de archivos.
- **Número de bloques libres**
- **Número de inodos libres**.
- **Primer inodo** (es el sistema raíz o directorio '/').

Puesto que hay muchas copias del superblock en caso de que alguna se estropee, se puede montar un sistema de archivos dañado desde una copia del superblock → <https://ngelinux.com/how-to-mount-a-filesystem-from-backup-superblock-location/>

Consultar información del sistema de archivos (Ext 2, 3, 4)

Consultar 'block size'; 'uuid', inodo, 'magic number', etc.:

- `dumpe2fs /dev/sda1 | grep -i superblock`
- `tune2fs -l /dev/sda1`

Consultar información del sistema de archivos

- `fsstat /dev/sda1` or `fsstat -o 2048 /dev/sda`
 - **Nota**: -o offset en sectores (para indicar dónde empieza en el disco el sistema de archivos)
- `mmls imatge.dd` → Muestra la tabla de particiones de una imagen (con autodetección)

Montar sistema de archivos de una imagen

- `sudo mount -o ro,loop,offset=1048576 -t ext2 myImage.dd /mnt/myImage`
 - **Nota**: offset en bytes (NO sectores) para indicar dónde comienza en el disco el sistema de archivos, para saltar particiones anteriores y bloques de metadatos.
- Montar Ext sin activar el journaling - <https://www.sans.org/blog/how-to-mount-dirty-ext4-file-systems/>

Otras herramientas

- Comprobación de disco: e2fsck
- Crear sistema de ficheros: mke2fs

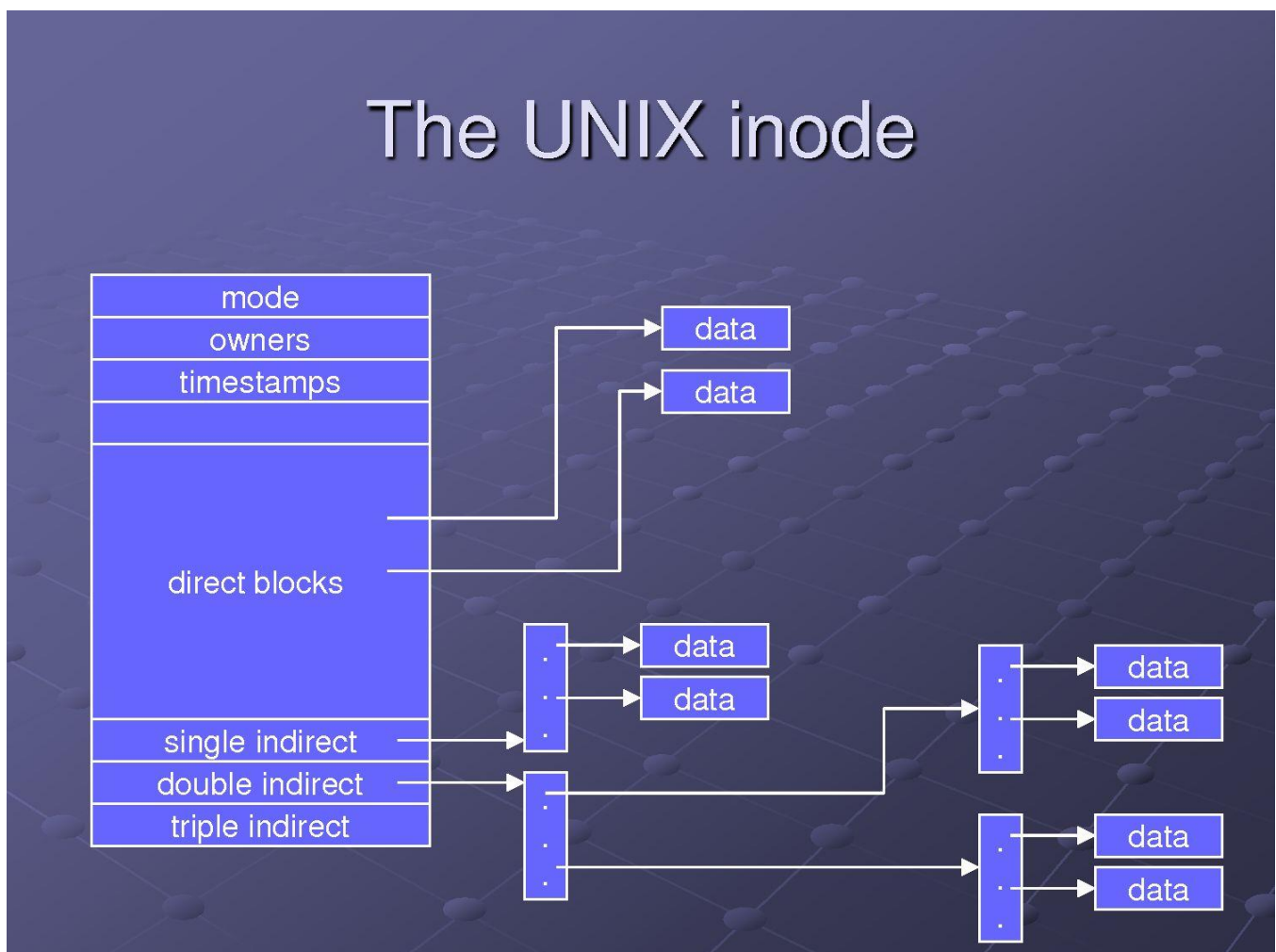
Más información sobre estas herramientas: <http://e2fsprogs.sourceforge.net/ext2.html>

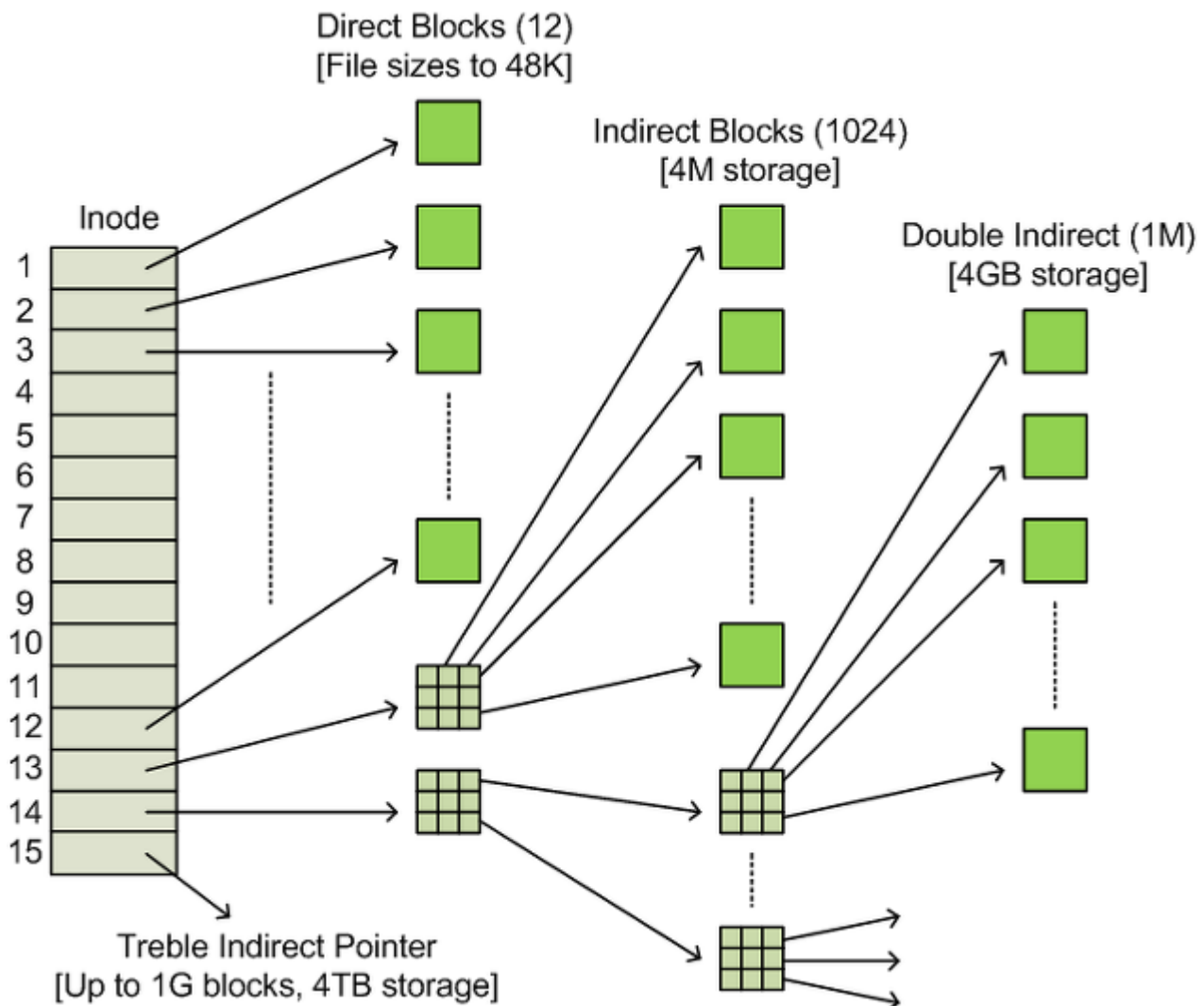
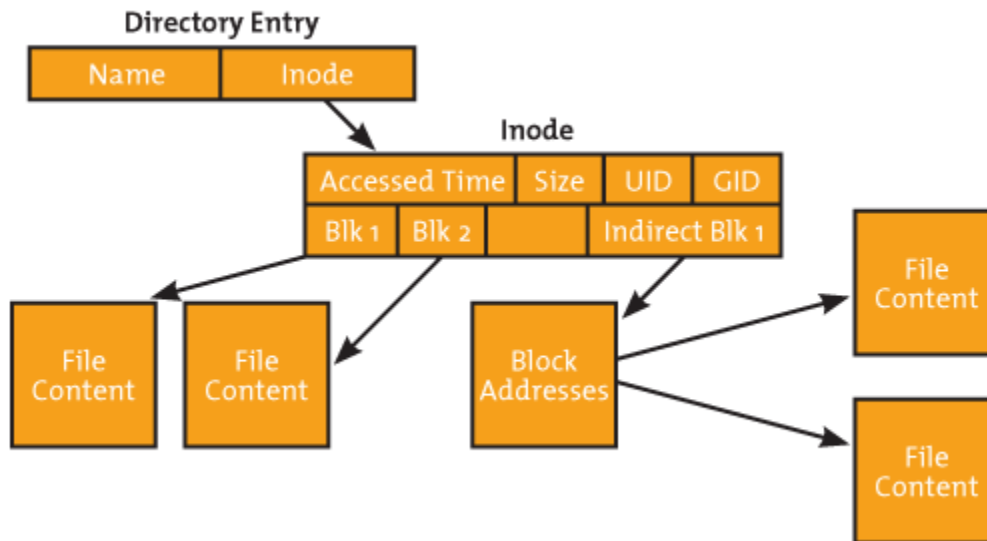
2.2 Los inodos

Para cada archivo hay un "inodo" que lo describe. Un Inodo es una estructura de datos que contiene metadatos con la siguiente información:

- Tipo de archivo
- Propietario
- bits de Permisos
- tiempo MAC (Modification, Access y Metadata Change).
- Enlaces al archivo (contador de links desde las entradas de directorio).
- Direcciones de los bloques de datos (directamente, indirectamente, doble indirectamente, triple indirectamente).

La organización de los bloques de archivos se puede observar en las siguientes imágenes.





2.3 ¿Qué ocurre cuando se crea un fichero?

Cada archivo:

- Tiene una o más entradas de directorio. Si tiene más de un enlace, cada enlace tiene su propia entrada de directorio pero compartiendo inodo.
- Tiene asignado un inodo por los metadatos y apuntadores a bloques
- Ocupa unos bloques para los datos,

Cuando creamos un archivo:

- Se elige un inodo libre del bitmap de inodos.
- Baja en uno el contador de inodos libres del superblock.
- Se hace una entrada en el directorio padre por el archivo (para poner el nombre).
- Se elige un bloque libre del bitmap de bloques.
- Se completan los datos del inodo.

2.4 ¿Cómo se elimina un fichero?

Cuando el contador de links de un inodo llega a 0:

- Los bitmap de bloques de datos se ponen a 0 (libres).
- El bitmap del inodo se pone a 0 (indica inodo libre).
- Se guarda la hora de borrado en el inodo.
- La entrada de directorio se marca como libre.

En ext3/ext4:

- El tamaño de archivo en el inodo se pone en 0.
- Los punteros de bloques de datos del inodo se ponen a 0.

2.5 Recuperación de archivos

Los datos todavía existen en el disco a pesar de haber borrado los archivos o incluso haber borrado la tabla de particiones:

- La información es recuperable hasta que se sobrescriben los bloques del disco.
- Cuanto mayor sea el disco, menos posibilidades hay de que el archivo sea sobrescrito.
- Si el disco es saneado (wiped), o sea sobrescrito con 0's, tenemos poco que hacer en la recuperación.

Se pueden recuperar archivos usando el journaling del FS. El "journaling" es un mecanismo por el que un sistema informático puede implementar transacciones (Ext3, Ext4):

```
extundelete /dev/xxx
```

Hay que tener cuidado cuando el sistema de archivos está sucio, o sea, se ha cerrado el sistema inesperadamente y quedan transacciones de disco para hacer guardadas en el journal.

Quizás puede interesar montar un sistema de archivos "sucio" sin que se apliquen las transacciones pendientes o se vuelvan atrás algunos últimos cambios que puedan ser interesantes por el forense - <https://www.sans.org/blog/how-to-mount-dirty-ext4-file-systems/>

Más información:

- <http://extundelete.sourceforge.net/>
- <https://deer-run.com/users/hal/EXT3FileRecovery.pdf>

3 Artefactos Linux

En el contexto de los sistemas Linux, se denomina artefactos a todos aquellos ficheros, registros y configuraciones que dejan rastro de la actividad del sistema y de sus usuarios. Estos artefactos se generan como resultado del uso normal del sistema operativo, de la ejecución de aplicaciones, de la autenticación de usuarios y de la interacción con recursos locales y remotos.

Desde el punto de vista de la administración de sistemas y, especialmente, del análisis forense, los artefactos Linux son de gran importancia, ya que permiten reconstruir eventos pasados, identificar accesos no autorizados, detectar modificaciones maliciosas y comprender el comportamiento de usuarios o atacantes. Incluso cuando un atacante intenta borrar evidencias, es habitual que queden restos en distintos puntos del sistema.

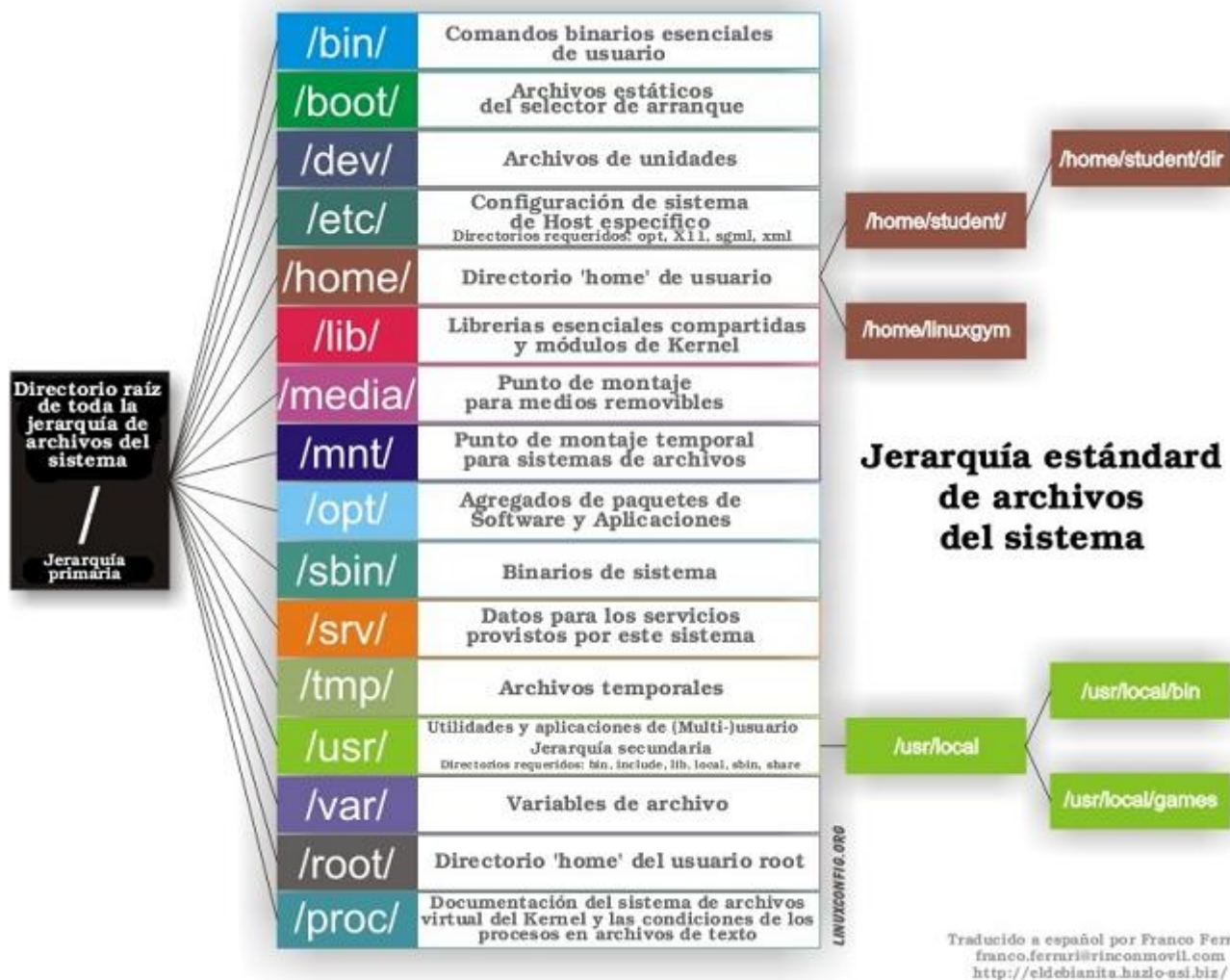
Linux organiza gran parte de estos artefactos siguiendo una jerarquía de directorios bien definida. Directorios como **/home**, **/etc**, **/var/log**, **/tmp** o **/root** contienen información crítica relacionada con datos de usuario, configuraciones del sistema, registros de actividad, archivos temporales y posibles cargas maliciosas. Además, los sistemas de ficheros mantienen metadatos asociados a cada archivo, como marcas de tiempo (timestamps), permisos y propietarios, que aportan información valiosa para el análisis.

Otro aspecto clave de los artefactos Linux es la información relacionada con usuarios y grupos, accesos remotos mediante SSH, historial de comandos, logs del sistema y de servicios, así como mecanismos de persistencia que pueden ser utilizados tanto legítimamente como con fines maliciosos. El estudio conjunto de todos estos elementos permite obtener una visión completa del estado y la historia de un sistema Linux.

En definitiva, el análisis de artefactos Linux constituye una base fundamental para tareas de auditoría, respuesta ante incidentes de seguridad y análisis forense digital, proporcionando evidencias técnicas que ayudan a comprender qué ha ocurrido en un sistema y cuándo ha sucedido.

3.1 La jerarquía de directorios Linux

Linux organiza todos los archivos del sistema siguiendo una **estructura jerárquica de directorios** que comienza en el directorio raíz, representado por `/`. Esta organización no es arbitraria, sino que sigue estándares como el *Filesystem Hierarchy Standard (FHS)*, cuyo objetivo es mantener coherencia entre distribuciones y facilitar la administración, el mantenimiento y el análisis del sistema.



Desde el punto de vista forense, comprender esta jerarquía es fundamental, ya que permite localizar rápidamente información relevante sobre el funcionamiento del sistema, la actividad de los usuarios y posibles acciones maliciosas.

En el directorio raíz `/` se encuentran los directorios esenciales del sistema. Entre ellos destaca `/bin`, que contiene los **binarios ejecutables básicos** necesarios para el funcionamiento del sistema y accesibles a todos los usuarios, como `ls`, `cp`, `mv` o `cat`. Estos comandos son imprescindibles incluso en modos de recuperación.

El directorio `/boot` alberga los archivos necesarios para el **arranque del sistema**, incluyendo el cargador de arranque y el kernel de Linux. **Cualquier modificación no autorizada en este directorio puede comprometer gravemente la seguridad del sistema.**

En **/dev** se representan los **dispositivos del sistema** como archivos especiales, tanto físicos (discos, particiones, terminales) como virtuales. Este enfoque permite que los dispositivos se gestionen como si fueran archivos convencionales.

El directorio **/etc** contiene los **archivos de configuración del sistema y de las aplicaciones**. Es uno de los directorios más sensibles desde el punto de vista forense, ya que **un atacante puede modificar configuraciones para mantener persistencia**, habilitar accesos o alterar el comportamiento del sistema.

En **/home** se encuentran los **directorios personales de los usuarios**, donde se almacenan documentos, configuraciones personales, historiales y claves SSH. Estos directorios suelen contener una gran cantidad de evidencias sobre la actividad del usuario.

El directorio **/lib** almacena las **bibliotecas compartidas** necesarias para que los programas ubicados en **/bin** y **/sbin** puedan ejecutarse correctamente.

Por su parte, **/lost+found** es utilizado por el sistema de archivos para recuperar archivos dañados o huérfanos tras errores de disco. Su contenido puede aportar información relevante tras fallos o apagados inesperados.

Finalmente, **/media** se utiliza como **punto de montaje para dispositivos extraíbles**, como memorias USB o discos externos, lo que puede ser útil para identificar el uso de medios externos.

Otros directorios relevantes bajo **/** incluyen **/opt**, destinado a **software opcional o de terceros** que no forma parte del sistema base. Suele utilizarse para aplicaciones instaladas manualmente.

El directorio **/proc** es un **sistema de archivos virtual** que proporciona información dinámica sobre los procesos y el estado del sistema. Aunque no contiene archivos reales, es muy útil para el análisis en vivo.

En **/root** se encuentra el **directorio personal del usuario root**, separado del resto de usuarios por motivos de seguridad.

El directorio **/sbin** contiene **binarios esenciales reservados al superusuario**, como herramientas de administración del sistema.

En **/srv** se almacenan los **datos servidos por el sistema**, por ejemplo, archivos ofrecidos por servidores web o FTP.

El directorio **/tmp** se utiliza para **archivos temporales**, normalmente borrados al reiniciar el sistema. Sin embargo, es frecuente que malware sin privilegios lo utilice para almacenar payloads o scripts temporales.

El directorio **/usr** agrupa la mayor parte de las **aplicaciones y utilidades multiusuario**, mientras que **/var** almacena los **datos variables**, como logs, colas, bases de datos y correos electrónicos.

Dentro de **/usr** encontramos una estructura similar a la del sistema base.

- En **/usr/bin** se almacenan aplicaciones generales accesibles a los usuarios normales, mientras que **/usr/sbin** contiene herramientas administrativas.
- El directorio **/usr/include** guarda los **archivos de cabecera** utilizados para el desarrollo de software, y **/usr/lib** las bibliotecas compartidas correspondientes.
- En **/usr/share** se almacenan **datos independientes de la arquitectura**, como documentación, iconos o archivos de idioma.
- El directorio **/usr/src** contiene **código fuente**, normalmente del kernel o de aplicaciones instaladas, y **/usr/local** se reserva para software específico del host, instalado fuera del gestor de paquetes.

El directorio **/var** es especialmente relevante en análisis forense, ya que contiene información que cambia constantemente.

- En **/var/log** se almacenan los **registros del sistema y de las aplicaciones**, fundamentales para reconstruir eventos.
- Otros subdirectorios incluyen **/var/cache**, con datos cacheados; **/var/crash**, con información sobre fallos del sistema; **/var/mail**, que contiene buzones de correo; y **/var/spool**, utilizado para colas de impresión o correo.
- El directorio **/var/run** (o **/run** en sistemas modernos) almacena información desde el último arranque, como procesos activos o usuarios conectados. Por último, **/var/tmp** contiene archivos temporales que **persisten entre reinicios**, a diferencia de **/tmp**.

Desde una perspectiva forense, los directorios más relevantes son **/home** y **/root**, por contener datos de usuario; **/etc**, por posibles configuraciones alteradas; **/var/log**, por los registros de actividad; y **/tmp** y **/var/tmp**, que pueden albergar restos de malware o scripts temporales.

3.2 Timestamps en Linux

Los timestamps o marcas de tiempo de los archivos proporcionan información esencial sobre cuándo se creó, modificó o accedió a un fichero. En sistemas de archivos antiguos como EXT3, no se almacena la fecha de creación. Sin embargo, en **EXT4** existe el atributo **ctime** o **btime**, que registra este dato.

Para consultar esta información se emplean comandos como **stat**, que permite obtener distintos timestamps:

- **Parámetro %w** para la creación (si existe)
- **Parámetro %y** para la modificación de contenido
- **Parámetro %z** para el cambio de atributos.

En análisis más profundos se puede utilizar **debugfs**, que accede directamente a la información del sistema de archivos.

Por defecto, el comando **ls -l** muestra el **mtime**, es decir, la fecha de última modificación del contenido. Con el comando **touch** es posible modificar manualmente los timestamps de acceso o modificación, lo que **representa una técnica habitual de antiforense**.

En tareas de investigación, herramientas como **find** permiten buscar archivos modificados recientemente (**-mtime**) o compararlos con un archivo de referencia (**-newer**). El parámetro **-printf** facilita la visualización detallada de rutas y marcas temporales.

Finalmente, es importante tener en cuenta el comportamiento del **atime**. Por motivos de rendimiento, muchos sistemas utilizan la opción **relatime**, lo que provoca que la fecha de último acceso se actualice solo de forma ocasional. Aun así, este timestamp puede resultar útil para identificar archivos o programas utilizados recientemente.

3.3 Información básica de un sistema Linux

La información relacionada con la distribución de Linux, la configuración de red, la fecha de instalación, la zona horaria y la gestión de usuarios constituye una parte esencial del análisis de un sistema Linux. Desde el punto de vista forense y de administración, estos datos permiten identificar el entorno en el que opera el sistema, reconstruir su historia y detectar posibles manipulaciones o configuraciones anómalas.

Información de la distribución

Las distintas distribuciones de Linux almacenan información sobre su versión y nombre en archivos ubicados dentro del directorio **/etc**. Habitualmente, estos archivos siguen el patrón **/etc/*-release**, como **/etc/lsb-release** en Debian y Ubuntu, o **/etc/redhat-release** en distribuciones basadas en Red Hat. Estos ficheros permiten conocer con precisión la distribución, su versión y, en algunos casos, el nombre en clave, lo cual resulta fundamental para contextualizar el sistema analizado y entender su comportamiento, así como las herramientas y servicios que pueden estar presentes.

Nombre del host (Hostname)

El nombre del sistema dentro de la red, conocido como hostname, se almacena generalmente en el archivo **/etc/hostname**. Este valor identifica de forma lógica a la máquina y puede aparecer reflejado en logs, configuraciones de red y sesiones remotas. Un cambio no autorizado del hostname puede ser indicativo de una manipulación del sistema o de un intento de ocultar su identidad dentro de una red.

Configuración de red

La configuración de las interfaces de red varía según la distribución de Linux. En sistemas basados en Debian y Ubuntu, esta información suele encontrarse en **/etc/network/interfaces**, mientras que en sistemas Red Hat, Fedora o CentOS se almacena en archivos dentro de **/etc/sysconfig/**. Estos ficheros definen parámetros clave como direcciones IP, puertas de enlace, interfaces activas y métodos de configuración (estática o dinámica). El análisis de estos archivos permite determinar cómo se conecta el sistema a la red y detectar configuraciones sospechosas o persistentes establecidas por un atacante.

Otra posibilidad es usar el comando **ipconfig** o **ip -a** en otros sistemas o sistemas antiguos.

Configuración de DNS

La resolución de nombres de dominio se configura principalmente en el archivo **/etc/resolv.conf**, donde se especifican los servidores DNS utilizados por el sistema. La modificación de este archivo puede permitir redirigir el tráfico a servidores controlados por un atacante, facilitando ataques de tipo man-in-the-middle o el control del tráfico de red. Por este motivo, su análisis resulta especialmente relevante.

Fecha de instalación del sistema (aproximación)

Linux no almacena de forma explícita la fecha y hora de instalación del sistema operativo. Por ello, es necesario aproximar esta información utilizando distintas fuentes indirectas.

Uno de los elementos más útiles es el directorio **/lost+found**, que se crea automáticamente al generar el sistema de archivos y rara vez se modifica posteriormente. **La fecha de creación de este directorio puede servir como una referencia aproximada del momento de instalación.**

Otra fuente relevante son las **claves de host SSH, como /etc/ssh/ssh_host_rsa_key**, que normalmente se generan durante el primer arranque del sistema. Su fecha de creación puede aportar una estimación bastante precisa del inicio de vida del sistema.

Asimismo, los logs de instalación, cuando existen, pueden proporcionar información directa sobre el proceso de instalación. En algunos sistemas, estos registros se conservan parcial o totalmente.

Herramientas como **tune2fs** o **dumpe2fs** permiten consultar metadatos del sistema de archivos, incluyendo la fecha de creación de este. Este dato, aunque no coincide exactamente con la instalación del sistema operativo, suele ser un indicador muy cercano en el tiempo.

Zona horaria del sistema

La zona horaria configurada en el sistema se almacena en el archivo **/etc/localtime**, el cual se encuentra en formato binario. Este archivo define cómo se interpretan y muestran las fechas y horas en el sistema.

Para analizar su contenido se puede emplear el comando **zdump**, que permite obtener información legible sobre la zona horaria configurada. Es importante tener en cuenta que, internamente, **los sistemas de archivos de Linux almacenan los timestamps en formato UTC**, y es el sistema el que los convierte a la zona horaria local al mostrarlos. Este detalle es crucial en análisis forense, ya que una interpretación incorrecta de la zona horaria puede llevar a errores en la reconstrucción temporal de los eventos.

Usuarios y grupos

La gestión de usuarios y grupos en Linux se basa en una serie de archivos críticos ubicados en **/etc**. El archivo **/etc/passwd** contiene la lista de usuarios del sistema, junto con información como el identificador de usuario (UID), el grupo primario, el directorio personal y la shell asignada.

El archivo **/etc/shadow** almacena los hashes de las contraseñas, así como información relacionada con la caducidad y políticas de estas. Su acceso está restringido por motivos de seguridad.

Por su parte, **/etc/group** define los grupos del sistema y los usuarios que pertenecen a cada uno de ellos como grupos secundarios.

El archivo **/etc/sudoers**, junto con los archivos del directorio **/etc/sudoers.d**, especifica qué usuarios o grupos tienen permisos para ejecutar comandos con privilegios elevados mediante **sudo**.

Ficheros abiertos por procesos del sistema

Podemos obtener un listado de archivos abiertos por los procesos con **lsdf** ("**ls open files**"). Permite pasar como parámetro el tipo de archivo a visualizar:

- archivo regular
- directorio
- archivo especial de bloques (discos)
- archivo especial de caracteres (teclado, consola...)
- referencia de código en ejecución
- biblioteca
- "stream"
- archivo de red (socket)

Ejemplos:

- **lsdf**: Todos los archivos que pertenecen a todos los procesos activos
- **lsdf -i [ipaddress]**: Conexiones Internet de una dirección IP determinada
- **lsdf -i 4 -a -p 1234**: Todos los archivos de red IPv4 en uso por el proceso PID 1234
- **lsdf /dev/hda3**: Todos los archivos abiertos del dispositivo /dev/hda3
- **lsdf /u/abe/foo**: Todos los procesos que tienen abierto el archivo /u/abe/foo

Criterios de análisis forense

Durante el análisis de usuarios y grupos es fundamental prestar atención a determinados indicadores de compromiso. **Entre ellos destacan las cuentas con shells no autorizadas, que deberían estar configuradas con /sbin/nologin o /bin/false.** También es importante **identificar usuarios con UID 0**, ya que este identificador otorga privilegios equivalentes a root y normalmente solo debería existir una única cuenta con este UID.

Por último, **se deben revisar los miembros del grupo sudo y las reglas definidas en sudoers**, con el fin de detectar usuarios que tengan privilegios administrativos sin una justificación legítima.

3.4 Análisis de logs

El análisis de logs constituye una de las fases más importantes en el estudio forense de un sistema Linux. Los registros del sistema permiten reconstruir eventos pasados, identificar accesos legítimos y no autorizados, detectar fallos, errores y actividades sospechosas, así como comprender el comportamiento general del sistema y de sus usuarios.

Logs en Linux

En la mayoría de las distribuciones Linux, los archivos de log se almacenan principalmente en el directorio **/var/log**. Estos ficheros suelen ser archivos de texto plano, lo que facilita su lectura y análisis mediante herramientas estándar como **cat**, **less**, **grep** o **awk**. No obstante, esta simplicidad también implica que, si un atacante obtiene privilegios suficientes, puede manipularlos o eliminarlos con relativa facilidad.

Los logs recogen información generada tanto por el sistema operativo como por los distintos servicios y aplicaciones en ejecución. Cada desarrollador decide qué eventos se registran, por lo que el contenido y nivel de detalle puede variar significativamente entre sistemas.

Inicios de sesión

Linux mantiene varios registros específicos relacionados con los inicios de sesión de los usuarios. El archivo **/var/log/wtmp** almacena el historial de **inicios y cierres de sesión exitosos**. Este fichero tiene formato binario y se interpreta mediante el comando **last**, que permite mostrar información como el usuario, el terminal, la dirección IP de origen y la duración de la sesión.

El archivo **/var/log/btmp** registra los intentos de **inicio de sesión fallidos**. También es un fichero binario y se analiza con el comando **lastb**. Su estudio puede revelar intentos de fuerza bruta o accesos no autorizados, aunque debe manejarse con precaución, ya que puede contener información sensible.

Por su parte, **/var/log/lastlog** guarda información sobre el **último inicio de sesión de cada usuario del sistema**. Se interpreta con el comando **lastlog** y resulta útil para identificar cuentas inactivas o accesos recientes.

Syslog

El **sistema de logging** centralizado en Linux es **gestionado habitualmente por el servicio rsyslogd**. Este demonio se encarga de recibir, clasificar y almacenar los mensajes de log generados por el kernel y por las aplicaciones.

Los logs gestionados por **syslog** **suelen rotarse periódicamente**, normalmente de forma semanal, comprimiendo los archivos antiguos y conservando un número limitado de copias. **Esto implica que la información histórica es finita y puede perderse con el tiempo si no se preserva.**

Entre las principales fuentes que escriben en **syslog** se encuentran:

- **auth y authpriv**, relacionadas con autenticación y seguridad.
- **kern**, que registra eventos del kernel y dispositivos.
- **cron**, que recoge la ejecución de tareas programadas.
- **daemon**, que agrupa mensajes de diversos servicios.

El análisis conjunto de estas fuentes permite obtener una visión global de la actividad del sistema.

Logs de servidores web

Los servidores web, como **Apache** y **Nginx**, generan sus propios registros, que suelen ubicarse en directorios como **/var/log/apache*** o **/var/log/nginx**. Estos logs son especialmente relevantes debido a la alta exposición de los servicios web a ataques externos.

En ellos se registra información como la dirección IP del cliente, la fecha y hora de la solicitud, el método HTTP utilizado (GET, POST, etc.), la ruta solicitada, el código de respuesta HTTP, el tamaño de la respuesta y, en muchos casos, el user-agent del cliente. Estos datos permiten identificar patrones de ataque, explotación de vulnerabilidades o la actividad de malware que utilice firmas reconocibles.

En la siguiente tabla podemos observar los artefactos más interesantes de los navegadores más comunes.

	Firefox	Chrome	Chromium
Cronología	/home/\$USER/.mozilla/firefox/\$PROFILE.default/places.sqlite	/home/\$USER/.config/google-chrome/Default/Preferences	/home/\$USER/.config/chromium/Default/Preferences
Preferenze	\$HOME/.mozilla/firefox/*.default	/home/\$USER/.config/google-chrome/Default/Preferences	/home/\$USER/.config/chromium/Default/Preferences
Cache	/home/\$USER/.mozilla/firefox/\$PROFILE.default/Cache/ /home/\$USER/.mozilla/firefox/\$PROFILE.default/cache2/	/home/\$USER/.cache/google-chrome/\$PROFILE/Cache/ /home/\$USER/.config/google-chrome/\$PROFILE/Cache/ /home/\$USER/.cache/google-chrome/\$PROFILE/Media Cache/ /home/\$USER/.config/google-chrome/\$PROFILE/Media Cache/ /home/\$USER/.config/google-chrome/\$PROFILE/Application Cache/Cache/ /home/\$USER/.config/google-chrome/\$PROFILE/GPUCache/ /home/\$USER/.cache/google-chrome/PnaclTranslationCache/	/home/\$USER/.cache/chromium/\$PROFILE/Cache/ /home/\$USER/.config/chromium/\$PROFILE/Cache/ /home/\$USER/.cache/chromium/\$PROFILE/Media Cache/ /home/\$USER/.config/chromium/\$PROFILE/Media Cache/ /home/\$USER/.config/chromium/\$PROFILE/Application Cache/Cache/ /home/\$USER/.config/chromium/\$PROFILE/GPUCache/ /home/\$USER/.cache/chromium/PnaclTranslationCache/

Uso de dispositivos externos

Como ya es sabido el conocimiento del uso d dispositivos externos también puede ser relevante en un análisis forense. En Linux podemos hacer uso del comando **lsusb** para conocer los **dispositivos conectados/desconectados a un sistema vivo**.

Si lo que necesitamos es averiguar esta información usando un adquisición de un sistema Linux, tendremos que acudir a los logs antes mencionados, usando comandos como por ejemplo **cat /var/log/syslog | grep USB**

Journaling

En los sistemas Linux, el **journaling** es un mecanismo propio de ciertos **sistemas de archivos** cuyo objetivo principal es **garantizar la consistencia e integridad del sistema** ante fallos inesperados, como apagados bruscos, cortes de energía o bloqueos del sistema.

Desde el punto de vista del **análisis forense**, el journaling cobra especial relevancia porque **registra operaciones pendientes** antes de que estas se escriban definitivamente en el disco. Este registro, denominado *journal*, puede contener información valiosa sobre **acciones recientes realizadas en el sistema**, incluso aunque no llegaran a completarse.

Funcionamiento básico

Cuando el sistema operativo necesita modificar el sistema de archivos (por ejemplo, crear, borrar o mover un archivo), primero **anota la operación en el journal**. Solo después de que esta operación quede registrada, se ejecuta el cambio real en el disco.

Si el sistema se apaga de forma inesperada:

- El sistema de archivos no queda en estado inconsistente.
- Al reiniciar, Linux revisa el journal y **completa o revierte las operaciones incompletas**.
- Se evita la necesidad de comprobaciones completas del sistema de archivos (fsck).
- **Importancia en un análisis forense**

En el ámbito forense, el journaling tiene varias implicaciones clave:

- **Persistencia de evidencias temporales:** el journal puede contener rastros de operaciones recientes sobre archivos, incluso si estos han sido eliminados o no se han escrito completamente.
- **Reconstrucción de eventos:** permite inferir **qué acciones se estaban realizando** justo antes de un fallo o apagado.
- **Alteración de evidencias:** el proceso automático de recuperación del journal al arrancar el sistema puede **modificar el estado del disco**, lo que obliga al perito a extremar las precauciones (por ejemplo, montar el sistema en modo solo lectura).
- **Sistemas de archivos con journaling relevantes en forense**

Los sistemas de archivos más habituales en Linux que incorporan journaling son:

- **ext4** (muy común en sistemas actuales)
- **XFS**
- **JFS**

Cada uno implementa el journaling de forma distinta, lo que influye en **qué información queda registrada y durante cuánto tiempo.**

Modos de journaling y su impacto forense

En sistemas como **ext4**, el modo de journaling configurado afecta directamente a la evidencia disponible:

- **Journal:** registra metadatos y datos. Es el modo más seguro y el más interesante desde el punto de vista forense.
- **Ordered (por defecto):** garantiza que los datos se escriban antes que los metadatos.
- **Writeback:** prioriza el rendimiento, pero puede dejar menos información útil para el análisis.

3.5 Servicio SSH

El servicio **SSH** es uno de los principales métodos de acceso remoto a sistemas Linux, por lo que sus artefactos tienen un gran valor forense.

El **archivo authorized_keys**, ubicado en el **directorio .ssh** del usuario, contiene las **claves públicas autorizadas** para autenticarse sin contraseña. Los atacantes suelen añadir sus propias claves para garantizar acceso persistente al sistema.

El **archivo known_hosts** almacena las **claves públicas de los sistemas remotos** a los que el usuario se ha conectado desde la máquina analizada. Su contenido puede revelar relaciones de confianza y conexiones previas.

Por último, el **archivo config** permite definir configuraciones específicas para conexiones SSH, incluyendo usuarios, puertos, claves y hosts. Su análisis puede descubrir accesos automatizados o configuraciones ocultas.

3.6 Historial de comandos y acceso a ficheros

El historial de comandos proporciona información directa sobre las acciones realizadas por un usuario en la línea de comandos. En shells como **bash** o **zsh**, estos historiales se almacenan en archivos como **.bash_history** o **.zsh_history**. Recuerda que esta información queda en el perfil del usuario.

Por defecto, estos ficheros **no incluyen marcas de tiempo**, lo que dificulta la reconstrucción cronológica de los eventos. **Además, son fácilmente editables y pueden ser manipulados con fines antiforenses.** También **es importante tener en cuenta que los comandos solo se guardan al cerrar la sesión.**

En cuanto al acceso a ficheros, **algunas aplicaciones mantienen sus propios historiales.** El comando **less** guarda los términos de búsqueda en **.lesshst**, mientras que **el editor vim registra archivos editados y posiciones de cursor en .viminfo.**

Los editores de texto también generan **copias de seguridad temporales**. Vim crea archivos con extensión .swp, mientras que **editores como nano añaden el carácter ~ al final del nombre del archivo**. Estas copias pueden revelar modificaciones recientes o intentos de ocultación.

3.7 Entorno gráfico

El entorno gráfico de Linux también genera artefactos relevantes. Los gestores de archivos almacenan información sobre los **archivos abiertos recientemente** en el fichero **~/.local/share/recently-used.xbel**, que tiene formato XML e incluye la aplicación utilizada.

Los archivos eliminados a través del entorno gráfico se mueven a la **papelera de reciclaje**, cuyos contenidos se almacenan en **~/.local/share/Trash/files**, mientras que los **metadatos asociados se guardan en ~/.local/share/Trash/info**. Esto permite recuperar información sobre archivos eliminados recientemente.

Los navegadores web, como Firefox y Chrome, almacenan historiales, cookies y otros datos en directorios como **~/.mozilla/firefox** y **~/.config/chromium**. Estos datos pueden analizarse con herramientas forenses similares a las utilizadas en otros sistemas operativos.

3.8 Backdoors

Las **backdoors** son mecanismos utilizados para mantener acceso no autorizado a un sistema. Entre las técnicas más comunes se encuentran la instalación de **web shells**, la modificación del servicio SSH para permitir accesos ocultos y el uso de cuentas del sistema que normalmente están bloqueadas.

Existen varios indicadores que pueden alertar de la presencia de un backdoor. Uno de los más graves es la existencia de **múltiples cuentas con UID 0**, lo que implica privilegios de administrador. Asimismo, cambios no autorizados en el archivo sudoers o en los grupos administrativos pueden indicar una escalada de privilegios.

3.9 Persistencia

Un atacante suele buscar mecanismos para garantizar que su acceso o malware persista tras reinicios del sistema. Uno de los métodos más habituales es instalarlo como **servicio del sistema**, especialmente en sistemas modernos que utilizan systemd. Estos servicios se definen en directorios como **/etc/systemd/system**.

Otra técnica común es el uso de **tareas programadas**, mediante cron o at. Los archivos relacionados se encuentran en **/etc/cron*** y en directorios de **spool** asociados.

Finalmente, la persistencia puede lograrse mediante **scripts de inicio** o archivos de configuración de usuario, como **~/.bashrc** o **~/.config/autostart**, así como mediante módulos del kernel, que representan un nivel avanzado de compromiso.

4 Triage en sistemas Linux

Como ya vimos en temas anteriores, el **traje forense** es una forma de adquisición de datos de un **sistema vivo** en **una investigación forense digital** cuyo objetivo es:

- Obtener **información rápida y relevante**
- **Priorizar** qué sistemas, discos o datos deben analizarse en profundidad

En Linux podemos utilizar alguna de las herramientas de triaje similares a las que utilizamos con Windows:

- **CyLR** (funcionamiento idéntico a Windows) - <https://github.com/orlikoski/CyLR>
- **LinTriage** (del mismo autor que Wintrriage) - <https://www.securizame.com/lintrriage/>
- **Velociraptor**: herramienta mucho más compleja y también más invasiva (al ejecutarla lanza su propio servidor web y está más orientada hacia la respuesta ante incidentes) - <https://docs.velociraptor.app/>

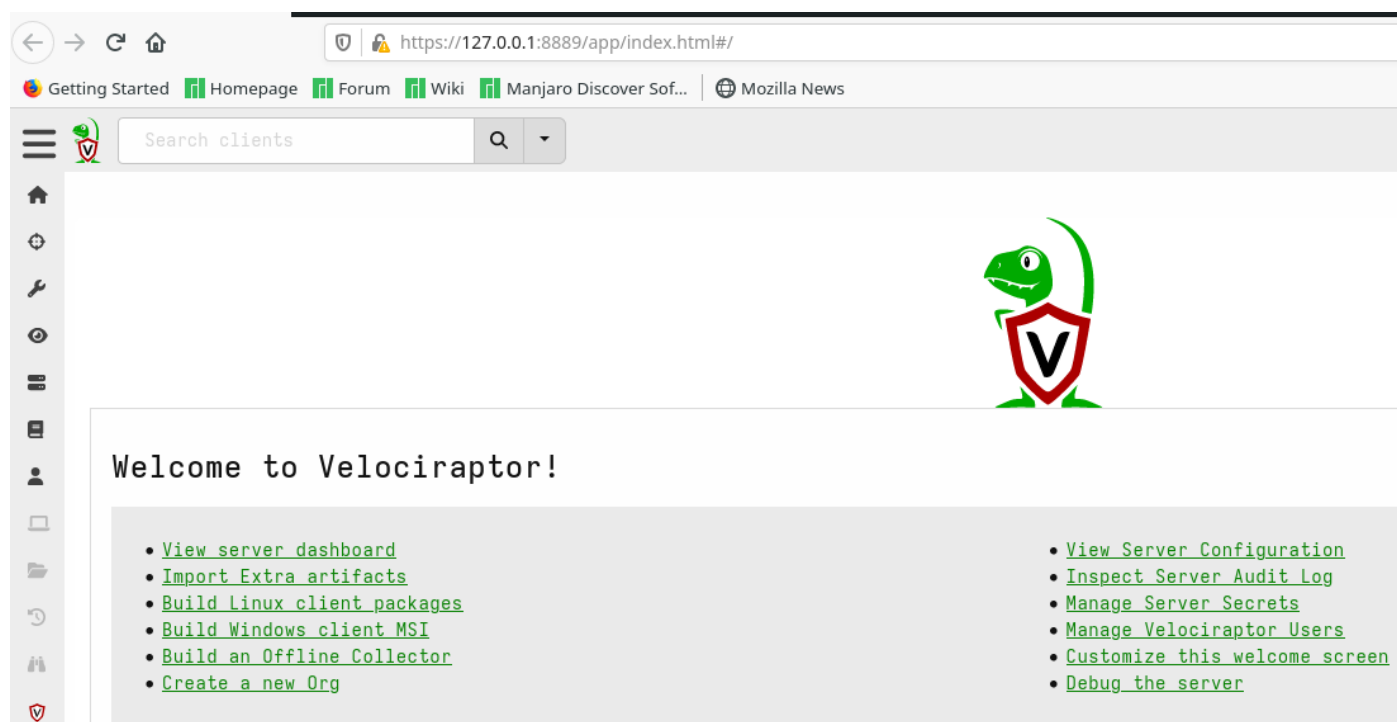
Uso básico de velociraptor

Dado que ya vimos CyLR y Wintrriage (prima de Lintrriage) anteriormente, vamos a ver el curioso funcionamiento de la herramienta **velociraptor** (solo para hacer triaje).

Es muy sencilla de utilizar. Basta con descargar la herramienta desde su web (apartado releases), copiarla en la máquina destino, darle permisos de ejecución (chmod +x) y ejecutarla así:

```
[osboxes@osboxes velociraptor]$ ./velociraptor-v0.75.6-linux-amd64 gui
```

Cuando termine de cargar se abrirá un navegador con la herramienta lista para usar:



Es una herramienta muy completa para realizar diferentes tareas. Para iniciar un triaje pulsaremos sobre el menú de hamburguesa y seleccionaremos la opción **“Hunt Manager”**. Una vez ahí pulsaremos el **botón +** y elegiremos la opción **“New Hunter”**:



New Hunt - Configure Hunt

Tags

Description

Expiry

Include Condition

Exclude Condition

Orgs

Hunt State

Hunt Tags (Type to create new Tag)

Hunt description

2026-01-07T14:40:49.741Z

Run everywhere

Run everywhere

All Orgs Select an org

☐ Start Hunt Immediately

Estimated affected clients 1

All known Clients

Configure Hunt
Select Artifacts
Configure Parameters
Specify Resources
Review
Launch

En parte de abajo veremos varias pestañas. Pulsaremos sobre la pestaña **Select Artifacts** y elegiremos los artefactos adecuados en función del triaje deseado (podemos elegir artefactos personalizados para Linux o artefactos genéricos):

Create Hunt: Select artifacts to collect

Linux.Sys.LastUserLogin
Linux.Sys.LogGrep
Linux.Sys.LogHunter
Linux.Sys.Maps
Linux.Sys.Pslist
Linux.Sys.SUID
Linux.Sys.Services
Linux.Sys.Users
Linux.Syslog.SSHLogin
Linux.Triage.ProcessMemory
Linux.Users.InteractiveUsers
Linux.Users.RootUsers
Linux.Utils.InstallDeb

Linux.Users.RootUsers

Type: client

Author: George-Andrei Iosif (@iosifache)

Detects users added in the sudo group.

Source

```

1 SELECT *
2 FROM foreach(
3   row={
4     SELECT *
5     FROM Artifact.Linux.Sys.Users()
6   },
7   query={
8     SELECT Fqdn AS Host,
9            User,
10           Description,
11           Uid,
12           Gid,
13           Homedir,
14           ...

```

Configure Hunt
Select Artifacts
Configure Parameters
Specify Resources
Review
Launch

Dejaremos el resto de las pestañas como están y pulsaremos sobre la pestaña “Launch”. Ya tenemos el triaje creado, pero falta ejecutarlo:

Para ejecutarlo seleccionaremos el triaje creado y pulsaremos el botón “Play”:

Run this hunt?

Are you sure you want to run these hunts?

0-1/1

10

State	Tags	HuntId	Description
⏸		H.D5AJEQTVI78A	

Y ya lo tendremos en modo ejecución. En el bloque de abajo podremos ver su evolución:

State	Tags	HuntId	Description	Created	Started	Expires	Scheduled
⌛		H.D5AK6PF64E0KK		2025-12-31T15:38:13.544Z	2025-12-31T15:38:30.451Z	2026-01-07T15:37:47.309Z	1

Overview	Requests	Clients	Notebook								
</											

Una vez termine (pasará al estado **Completed**), podremos descargar los resultados desde la pestaña **Overview**:

Overview	Requests	Clients	Notebook
Overview			
Artifact Names	🔗 Linux.Network.Netstat 🔗 Linux.OSQuery.Generic 🔗 Linux.Proc.Modules 🔗 Linux.Ssh.AuthorizedKeys 🔗 Linux.Ssh.KnownHosts 🔗 Linux.Sys.BashHistory 🔗 Linux.Sys.BashShell 🔗 Linux.Sys.Crontab 🔗 Linux.Sys.Groups 🔗 Linux.Sys.LogGrep		
Results			
Total scheduled	1		
Finished clients	1		
Download Results	👍 🗑️		
Available Downloads	Full Download Summary Download Summary (CSV Only) Summary (JSON Only)		
📄 H.D5AK6PF64E0KK			
Uncompressed	592		
Compressed	107 Kb		
Container Files	87		

Después pulsamos por el enlace generado y se descargarán los resultados en un fichero zip.



H.D5AK6PF64EO
KK.zip

Si abrimos el fichero comprimido podemos ver que los resultados los muestra en diferentes formatos, sin respetar la ubicación de los ficheros originales:

 H.D5AK6PF64EOKK.zip

Archive Edit View Help

  Open   Extract   

    Location:  /osboxes-C.1c0aed7472ba241f/results/

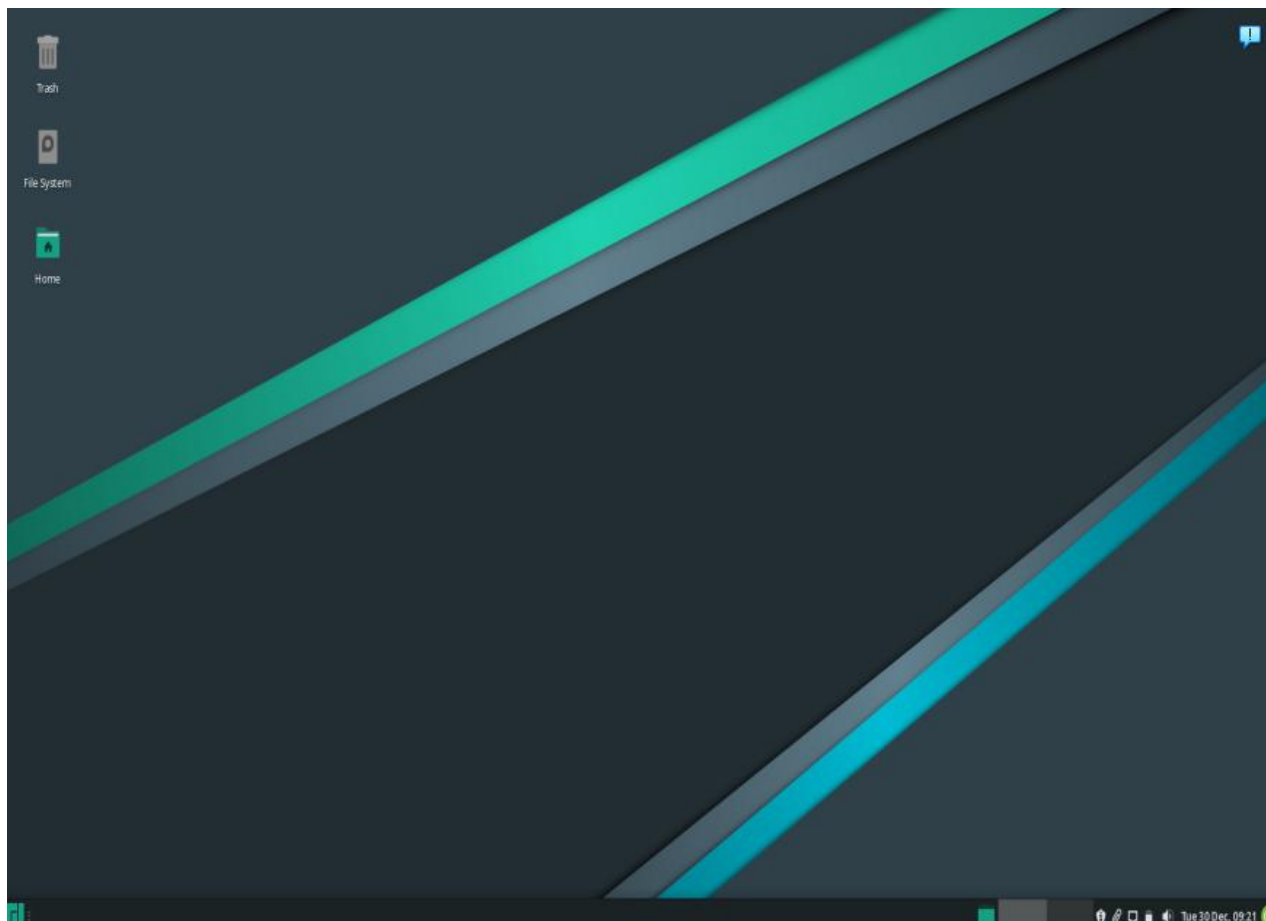
Name	Size	Type	Date Modified
 Linux.Mounts.csv	3.5 kB	CSV document	31 December 1969, 19:00
 Linux.Mounts.json	4.6 kB	JSON docum...	31 December 1969, 19:00
 Linux.Network.Netstat%2FTCP4,...	4.6 kB	CSV document	31 December 1969, 19:00
 Linux.Network.Netstat%2FTCP4,j...	5.9 kB	JSON docum...	31 December 1969, 19:00
 Linux.OSQuery.Generic.csv	250 bytes	CSV document	31 December 1969, 19:00
 Linux.OSQuery.Generic.json	300 bytes	JSON docum...	31 December 1969, 19:00
 Linux.Proc.Modules.csv	3.7 kB	CSV document	31 December 1969, 19:00
 Linux.Proc.Modules.json	8.3 kB	JSON docum...	31 December 1969, 19:00
 Linux.Sys.BashHistory.csv	1.5 kB	CSV document	31 December 1969, 19:00
 Linux.Sys.BashHistory.json	2.1 kB	JSON docum...	31 December 1969, 19:00
 Linux.Sys.BashShell.csv	1.2 kB	CSV document	31 December 1969, 19:00
 Linux.Sys.BashShell.json	1.3 kB	JSON docum...	31 December 1969, 19:00

30 objects (172.5 kB)

5 Ejemplo práctico

En este punto veremos como llevar a la práctica, tanto con un sistema vivo como muerto, el análisis forense de un sistema Linux.

Para ello descargamos una máquina virtual de <https://osboxes.org> , la importamos y la iniciamos:



5.1 Obtención de información de un sistema vivo

Abrimos un terminal y probamos los diferentes comandos que podemos ejecutar para obtener información de un sistema vivo:

Obtención de la fecha de instalación del sistema:

```
[osboxes@osboxes ~]$ ls -la /etc/ssh
total 588
drwxr-xr-x  2 root root   4096 Jan  3  2021 .
drwxr-xr-x 101 root root   4096 Dec 30 09:17 ..
-rw-r--r--  1 root root 577834 Oct 26  2020 moduli
-rw-r--r--  1 root root   1531 Oct 26  2020 ssh_config
-rw-r--r--  1 root root   3140 Oct 26  2020 sshd_config
```

Obtención de la información técnica del sistema:

```
[osboxes@osboxes ~]$ cat /etc/*-release
Manjaro Linux
DISTRIB_ID=ManjaroLinux
DISTRIB_RELEASE=20.2.1
DISTRIB_CODENAME=Nibia
DISTRIB_DESCRIPTION="Manjaro Linux"
Manjaro Linux
NAME="Manjaro Linux"
ID=manjaro
ID_LIKE=arch
BUILD_ID=rolling
PRETTY_NAME="Manjaro Linux"
ANSI_COLOR="32;1;24;144;200"
HOME_URL="https://manjaro.org/"
DOCUMENTATION_URL="https://wiki.manjaro.org/"
SUPPORT_URL="https://manjaro.org/"
BUG_REPORT_URL="https://bugs.manjaro.org/"
LOGO=manjarolinux
```

¿Cuánto tiempo está el sistema en marcha?:

```
[osboxes@osboxes ~]$ uptime
09:45:08 up 28 min, 1 user, load average: 0.00, 0.01, 0.05
```

¿El sistema tiene el reloj y zona horaria correcta?:

```
[osboxes@osboxes ~]$ date
Tue Dec 30 09:50:50 AM EST 2025
```

Obteniendo información de la configuración de red:

Configuración de red local:

```
[osboxes@osboxes ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:2a:7c:6a brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 84138sec preferred_lft 84138sec
    inet6 fd00::e97a:c512:77:e073/64 scope global dynamic noprefixroute
        valid_lft 86148sec preferred_lft 14148sec
    inet6 fe80::2eb4:bea9:5982:f4c7/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Obteniendo dispositivos conectados por usb:

```
[osboxes@osboxes ~]$ lsusb
Bus 002 Device 002: ID 0951:176b Kingston Technology XS2000
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 002: ID 80ee:0021 VirtualBox USB Tablet
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

Fichero abiertos por procesos en ejecución:

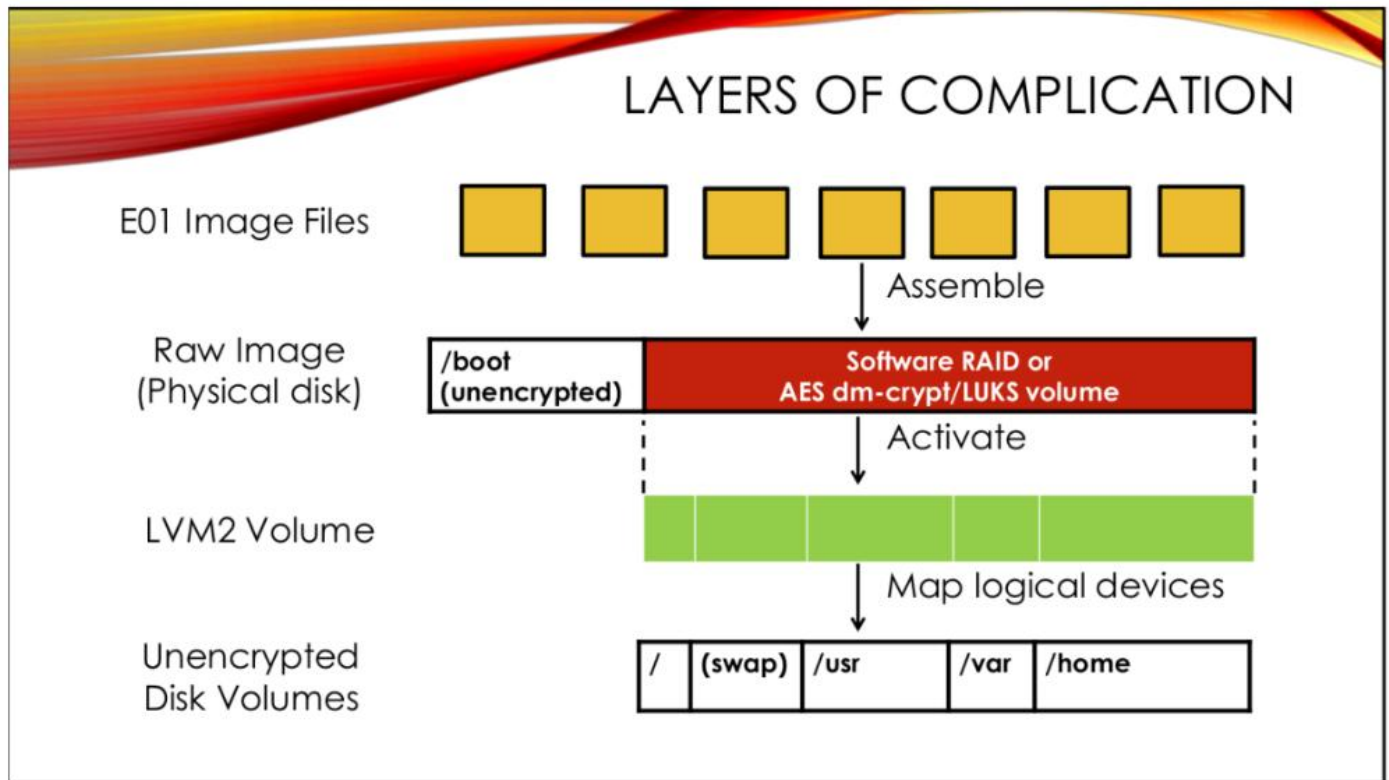
```
[osboxes@osboxes ~]$ lsof
COMMAND  PID  TID  TASKCMD  USER  FD  TYPE  DEVICE  SIZE/OFF  NODE  NAME
systemd  826  systemd  osboxes  cwd    DIR    8,1    4096      2  /
systemd  826  systemd  osboxes  rtd    DIR    8,1    4096      2  /
systemd  826  systemd  osboxes  txt    REG    8,1  1714224 13117406 /usr/lib/systemd/systemd
systemd  826  systemd  osboxes  mem    REG    8,1   42960 13113559 /usr/lib/libffi.so.7.1.0
systemd  826  systemd  osboxes  mem    REG    8,1  153560 13113593 /usr/lib/libgpg-error.so.0.30.0
systemd  826  systemd  osboxes  mem    REG    8,1   22216 13113497 /usr/lib/libattr.so.1.1.2448
systemd  826  systemd  osboxes  mem    REG    8,1  100096 13113821 /usr/lib/libz.so.1.2.11
systemd  826  systemd  osboxes  mem    REG    8,1   26424 13113509 /usr/lib/libcap-ng.so.0.0.0
systemd  826  systemd  osboxes  mem    REG    8,1  158744 13113727 /usr/lib/libpthread-2.32.so
systemd  826  systemd  osboxes  mem    REG    8,1   18608 13113530 /usr/lib/libdl-2.32.so
systemd  826  systemd  osboxes  mem    REG    8,1  157456 13113642 /usr/lib/liblzma.so.5.2.5
systemd  826  systemd  osboxes  mem    REG    8,1  853696 13113822 /usr/lib/libzstd.so.1.4.5
systemd  826  systemd  osboxes  mem    REG    8,1 1261048 13113698 /usr/lib/libp11-kit.so.0.3.0
systemd  826  systemd  osboxes  mem    REG    8,1 2986744 13113517 /usr/lib/libcrypto.so.1.1
systemd  826  systemd  osboxes  mem    REG    8,1  136888 13113641 /usr/lib/liblz4.so.1.9.3
systemd  826  systemd  osboxes  mem    REG    8,1   35040 13113617 /usr/lib/libip4tc.so.2.0.0
systemd  826  systemd  osboxes  mem    REG    8,1 1180096 13113572 /usr/lib/libgcrypt.so.20.2.8
systemd  826  systemd  osboxes  mem    REG    8,1  186112 13113516 /usr/lib/libcrypt.so.2.0.0
systemd  826  systemd  osboxes  mem    REG    8,1   34528 13113510 /usr/lib/libcap.so.2.45
systemd  826  systemd  osboxes  mem    REG    8,1  326512 13113501 /usr/lib/libblkid.so.1.1.0
systemd  826  systemd  osboxes  mem    REG    8,1   38704 13113484 /usr/lib/libacl.so.1.1.2253
systemd  826  systemd  osboxes  mem    REG    8,1 2159552 13113505 /usr/lib/libc-2.32.so
```

5.2 Análisis de imagen de un sistema Linux

El primer problema al que nos enfrentaremos es montar la imagen. La mejor solución para ello es montarla en un sistema Linux, aunque también podríamos hacer uso del subsistema Linux que Windows incorpora desde su versión 10.

Aun así, las imágenes pueden darnos algunos problemas:

- Las imágenes no en bruto (E01 por ejemplo) pueden dar más problemas que las de formato RAW
- Podemos encontrarnos con particiones que usen volúmenes lógicos (**LVM**) para organizar las particiones físicas
- Que un apagado brusco nos lleve a una imagen sucia
 - Quedan transacciones pendientes de realizar
 - Más difícil de montar la imagen, pero posible
- Que el disco esté cifrado (p.ej.: dm-crypt - LUKS)
- Que haya un RAID software



Para intentar solventar todos estos problemas y otros nos ayudaremos de las ewf-tools - <https://github.com/libyal/libewf>

Las podremos instalar en cualquier sistema Linux así:

```
$ sudo apt install libewf2 ewf-tools
```

Y con esta herramienta empezamos a montar la imagen (más cómodo con un terminal de superusuario). Por ejemplo, vamos a montar una imagen de un Linux Mate en formato E01:

```
$ sudo su
```

Creamos una carpeta para montar la imagen:

```
# mkdir /tmp/disco01
```

Montamos la imagen con ewfmount:

```
# ewfmount /media/sf_Downloads/mate-MUS22.E01 /tmp/disco01
ewfmount 20140816
```

El resultado será un fichero (ewf1) del mismo tamaño que la imagen

```
# ls -lisa /tmp/disco01
total 0
1 0 dr-xr-xr-x  2 root root      0 Jan  2 14:39 .
1 0 drwxrwxrwt 13 root root    340 Jan  2 14:39 ..
2 0 -r--r--r--  1 root root 85899345920 Jan  2 14:39 ewf1
```

A continuación, podemos ver el tipo de partición y los sistemas de ficheros albergados:

```
# cd /tmp/disco01

(root@kali)-[/tmp/disco01]
# file ./ewf1
./ewf1: DOS/MBR boot sector

(root@kali)-[/tmp/disco01]
# fdisk -l ./ewf1
Disk ./ewf1: 80 GiB, 85899345920 bytes, 167772160 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x2be33195

Device      Boot   Start        End    Sectors   Size Id Type
./ewf1p1    *          2048    1050623    1048576   512M  b W95 FAT32
./ewf1p2                1052670 166717442 166717442   79.5G  5 Extended
./ewf1p5                1052672 166717440 166717440   79.5G  83 Linux

(root@kali)-[/tmp/disco01]
# mmls ./ewf1
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

    Slot      Start          End          Length      Description
000:  Meta      00000000000    00000000000    00000000001  Primary Table (#0)
001:  _____ 00000000000    0000002047    0000002048    Unallocated
002:  000:000    0000002048    0001050623    0001048576    Win95 FAT32 (0x0b)
003:  _____ 0001050624    0001052671    0000002048    Unallocated
004:  Meta      0001052670    0166717442    0166717442    DOS Extended (0x05)
005:  Meta      0001052670    0001052670    00000000001    Extended Table (#1)
006:  001:000    0001052672    0166717440    0166717440    Linux (0x83)
007:  _____ 0167770112    0167772159    0000002048    Unallocated
```

En este caso la partición que nos interesa es la de tipo Linux. Para poder dar el siguiente paso (montarla en un dispositivo loop **/dev/loopx**) debemos saber el tamaño de bloque (observa la captura, es de **512**) y el inicio de la partición (**1052672**).

Con esta información asignaremos la partición que nos interesa del fichero **ewf1** a un dispositivo loop con la herramienta **losetup** (con la opción **-a** comprobamos la correcta asignación al loop0):

```
# losetup -rf -o $((512*1052672)) ./ewf1

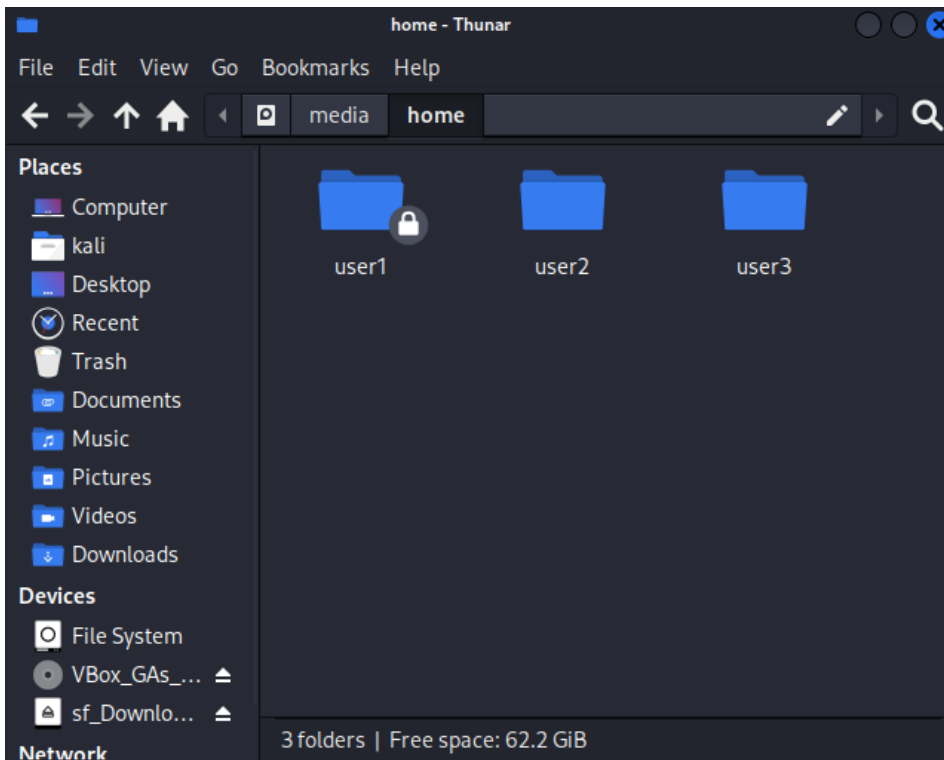
(root@kali)-[/tmp/disco01]
# losetup -a
/dev/loop0: [0048]:2 (/tmp/disco01/ewf1), offset 538968064
```

Y ya solo nos quedaría hacer el montaje en cualquier lugar:

```
# mount /dev/loop0 /media
mount: /media: WARNING: source write-protected, mounted read-only.

(root@kali)-[/tmp/disco01]
# ls /media
bin boot cdrom dev etc home lib lib32 lib64 libx32 lost+found media mnt opt proc root run sbin snap srv swapfile sys tmp usr var
```

Ya podremos acceder a las carpetas de usuario (por ejemplo) con el explorador de archivos:



A partir de este punto ya podríamos empezar la investigación forense de esta partición de la imagen.

Nota: Una alternativa a la herramienta losetup es directamente usar el comando mount con el desplazamiento calculado ($512 \times 1052672 = 538968064$):

```
(root@kali)-[/tmp/disco01]  
# mount -o ro,loop,offset=538968064 ./ewf1 /media
```

Revisa el manual de losetup para conocer como desvincular la partición al dispositivo de loop **/dev/loopx**